



第二讲 基础IO和逻辑控制

责任助教：胡云淏



01. 引入：MCU基本原理

02. ESP32-S3简介

03. GPIO的原理和使用

04. 中断的原理和使用



01. 引入：MCU基本原理



1.1. 程序如何控制硬件

一行printf()如何变为屏幕上的字符?

软件域
程序-处理器

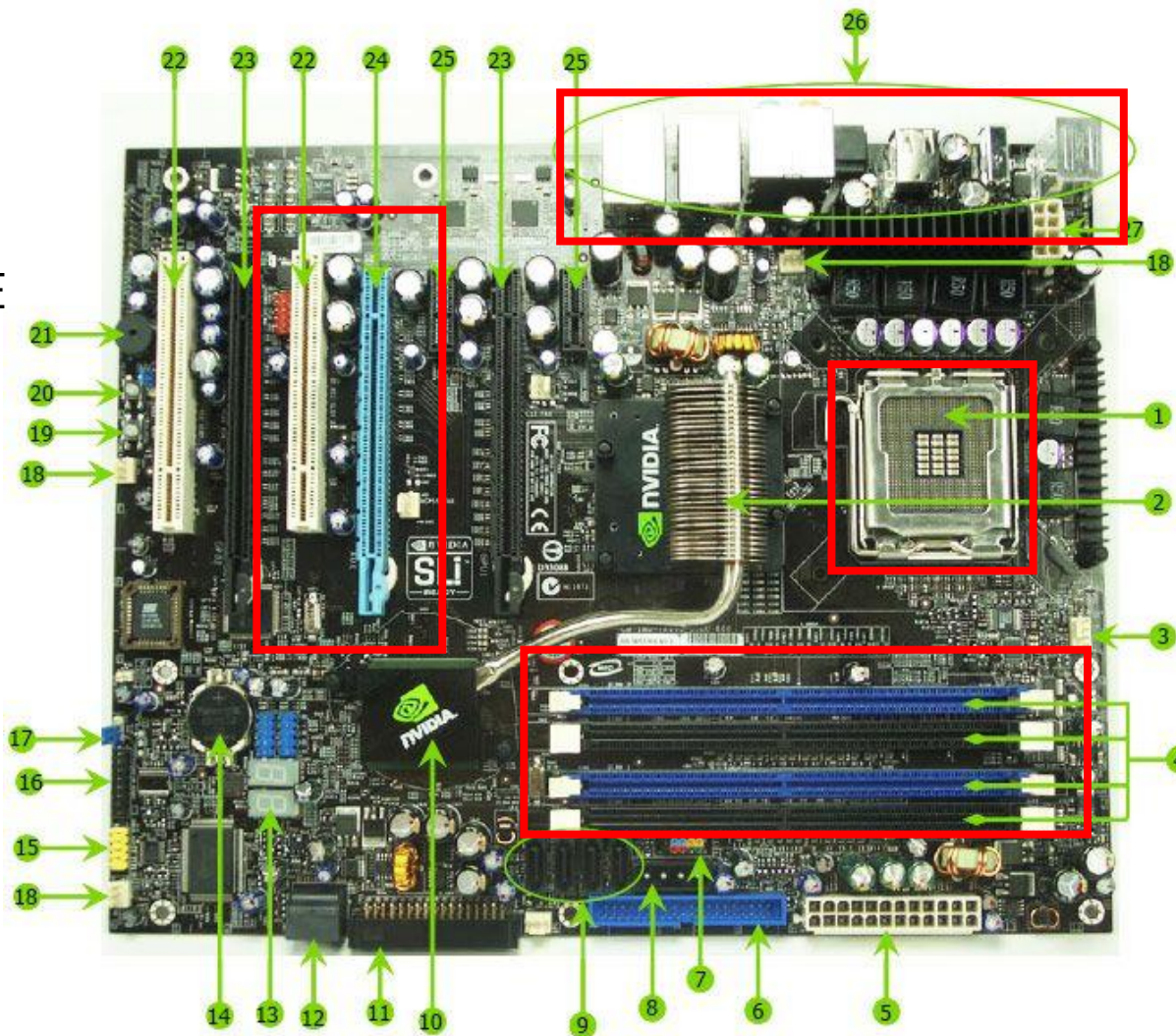
连接器 (linker) 编译器 (compiler) OS图形子系统
printf() → C标准库/Win32 API → 系统调用 (SYSCALL) → 显卡驱动
PCIe总线/
DDR/GDDR总线

硬件域
比特-逻辑
电势-电路

写入内存/显存
HDMI/DP线
显示器内部控制器
驱动电路
显示器上的像素

1.1. 程序如何控制硬件

2. PCIe接口
用于连接显卡，图形在这里渲染



4. 多种I/O接口
连接显示器、硬盘、键盘、鼠标、网线等
数据经此到达显示器

1. CPU插槽
用户程序和OS运行在这里

3. DDR接口
用于连接内存，显示器每个像素的状态实际上存储在这里

PC主板的基本结构

1.2. MCU基本结构

MCU (Micro Controller Unit) = 经过高度简化、高度浓缩的PC

对应CPU，执行程序

处理器核

多种外设
Peripherals

相当于网卡、键盘、USB接口等设备的大集合，是MCU的“眼睛”和“双手”

SRAM对应内存，存储临时数据
Flash对应硬盘，储存程序和长期数据，掉电不丢失

SRAM &
Flash

时钟树

为各种模块提供时钟信号，确保模块之间可以协调一致

全部集成与一块芯片中，即插即用

1.2. MCU基本结构

什么是外设 (Peripherals) ?

外设其实是“内部设备”：“外”是指在处理器核的外部，外设本身被集成在芯片内部。

外设是计算机系统的核心组件（处理器、内存）外，通过接口连接的外部设备的总称。

MCU中常见的外设：

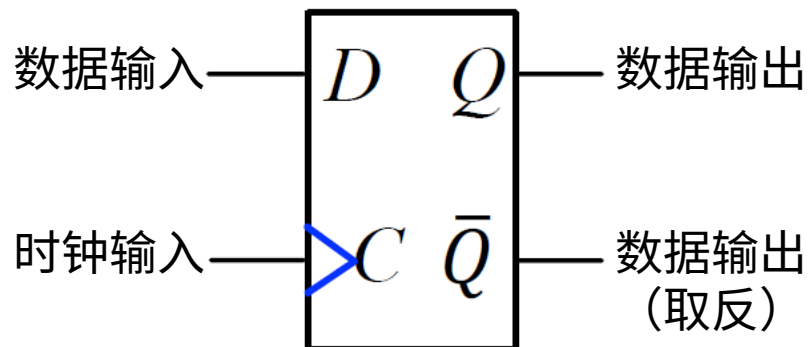
1. GPIO：通过引脚，读取电平或输出电平，让MCU具有输入输出能力
2. ADC：将模拟输入转化为数字输入，让MCU可以读取模拟信号
3. UART：输出符合UART的通信信号，使得MCU可以与其他设备通信
4. 定时器 (TIM)：计时，使得MCU可以定时执行任务，或者输出周期性信号

1.2. MCU基本结构

处理器与外设的区别

	处理器	外设
可编程性	可编程性强，可以执行通用任务	可编程能力极低，只能执行特定任务
核心职责	处理程序中的核心逻辑和数值运算	负责芯片与外界物理世界的输入输出，以及定时、加密等
运行频率	通常最快（ESP32S3中为240MHz），速度直接决定MCU的性能	通常较慢（UART常设置为80MHz），任务简单因此无需高速，低速时钟可以降低功耗
C语言编程	绝大部分程序（如if-else, for, 数学运算）都在处理器中执行	需要使用指针操作或库函数间接控制

1.3. MCU如何执行“printf”



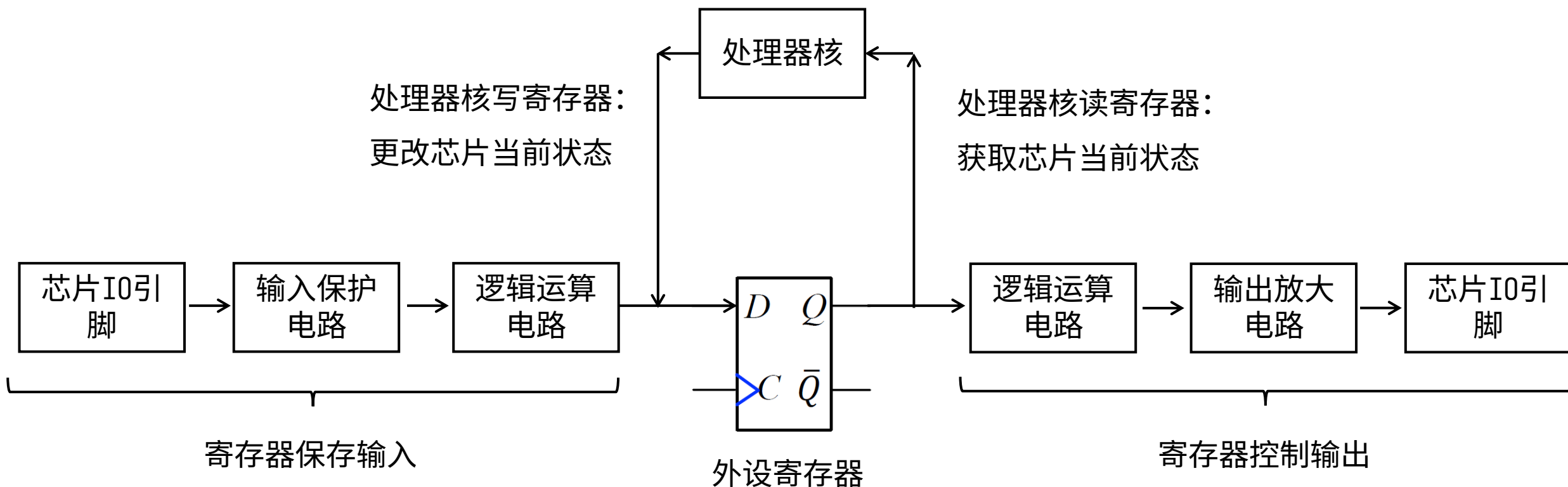
D	C	Q^+	$\bar{Q}^+$
0	↑	0	1
1	↑	1	0
0/1	0/1	Q	$\bar{Q}$

DFF (D flip-flop): 一个能保存0和1的电路

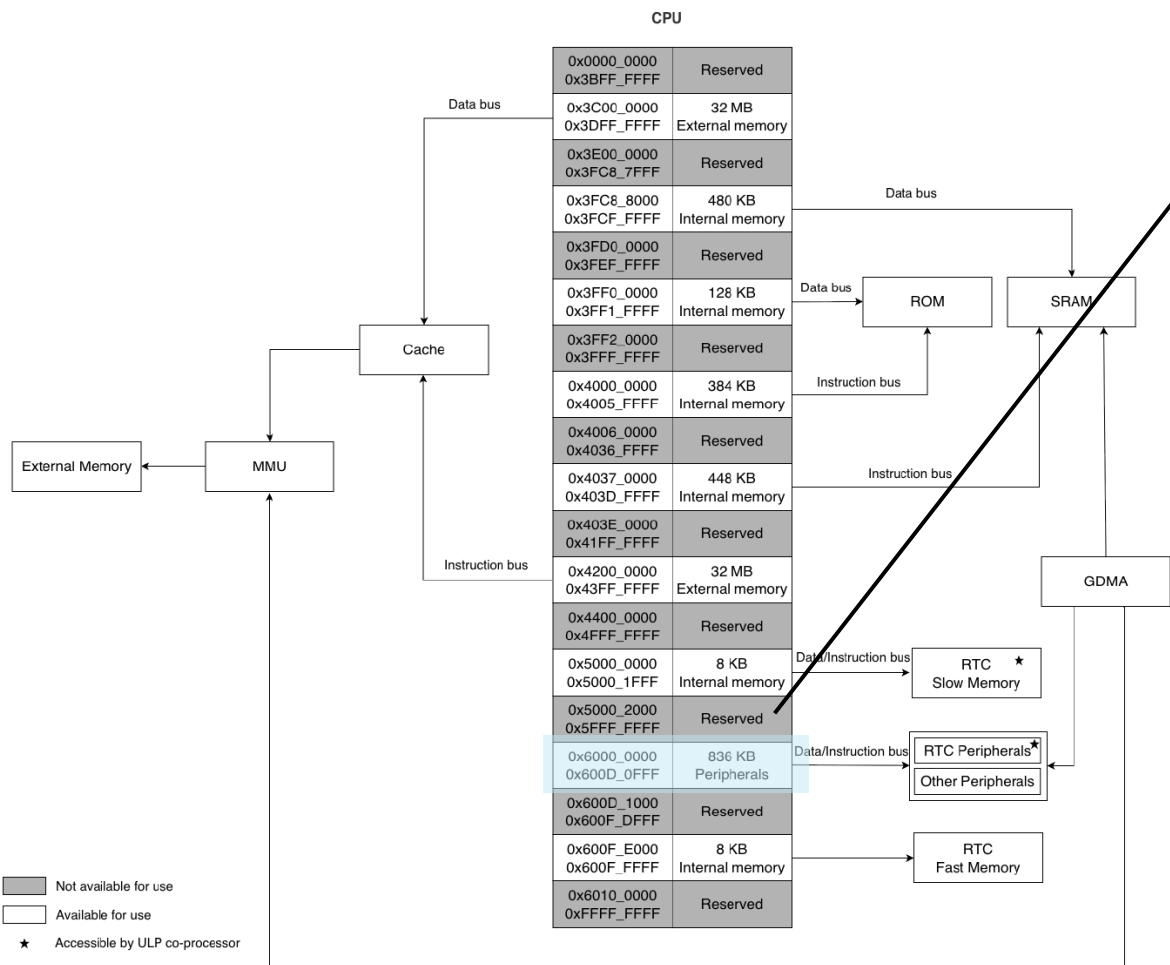
在平时保持Q不变，
在时钟上升沿读取D并保存

多个DFF和控制电路组成**寄存器 (Register)**，可以保存多位数据

1.3. MCU如何执行“printf”



1.3. MCU如何执行“printf”



特定地址对应外设寄存器

在C语言中，可以使用**指针**访问特定地址的寄存器

```
#define SENSOR_ADDR ((volatile uint32_t *)0x114514)

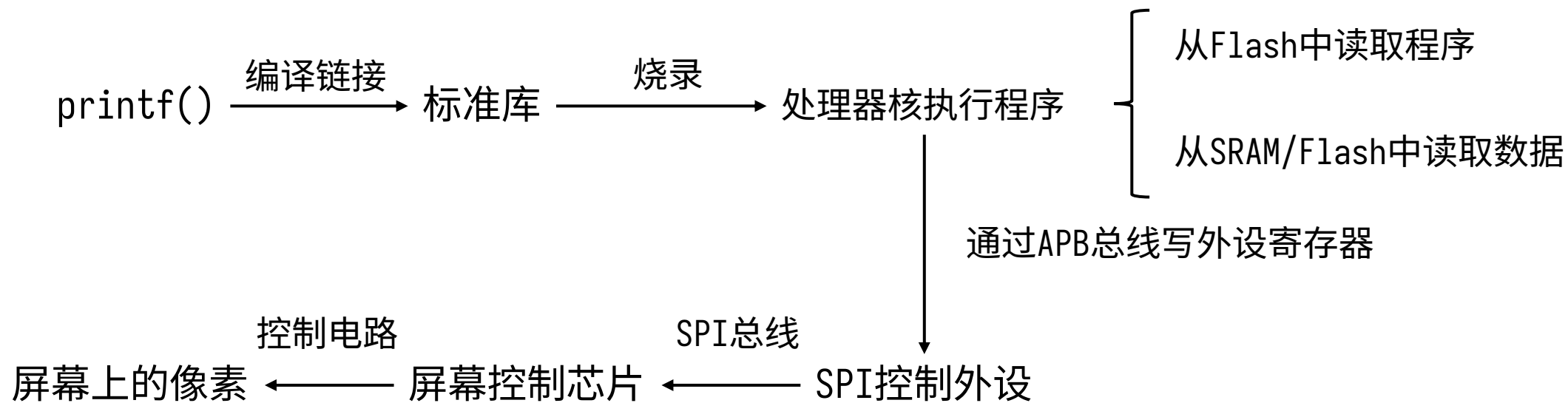
int main()
{
    while (1)
    {
        int sensor_data = *SENSOR_ADDR;
    }
}
```

软件视角：一段无用的死代码

硬件视角：寄存器中的数值随时可能被外部硬件修改

图 4.2-1. 系统结构与地址映射结构

1.3. MCU如何执行“printf”



核心差异:

1. 大部分操作都由MCU芯片直接完成
2. 没有OS! 对于硬件有100%的控制权

1.4. 什么是“嵌入式开发”

```
PS D:\code\HydroSQL> git diff 662291e9971080aadfc741e3fd17ffdd53eb2fd2 58a3af16a29bd86d4041d5aad73a43d521a081e
diff --git a/Doxyfile b/Doxyfile
index 49105bc..371f47d 100644
--- a/Doxyfile
+++ b/Doxyfile
@@ -42,7 +42,7 @@ DOXYFILE_ENCODING      = UTF-8
# title of most generated pages and in a few other places.
# The default value is: My Project.

-PROJECT_NAME      = "My Project"
+PROJECT_NAME      = "HydroSQL"

# The PROJECT_NUMBER tag can be used to enter a project or revision number. This
# could be handy for archiving the generated documentation or if some version
@@ -74,7 +74,7 @@ PROJECT_ICON      =
# entered, it will be relative to the location where Doxygen was started. If
# left blank the current directory will be used.

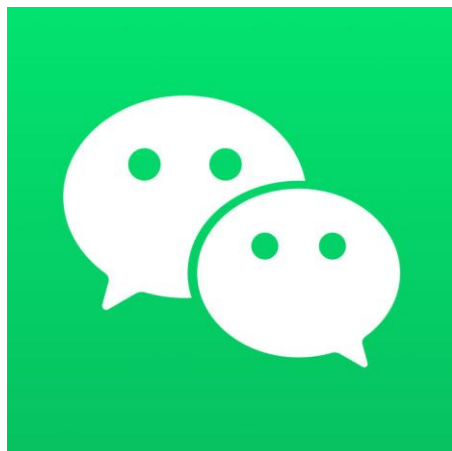
-OUTPUT_DIRECTORY  =
+OUTPUT_DIRECTORY  = ./docs

# If the CREATE_SUBDIRS tag is set to YES then Doxygen will create up to 4096
# sub-directories (in 2 levels) under the output directory of each output format
@@ -534,7 +534,7 @@ TIMESTAMP      = NO
# normally produced when WARNINGS is set to YES.
# The default value is: NO.

-EXTRACT_ALL       = NO
+EXTRACT_ALL       = YES
```



程序
↓
外设
↓
外围电路



1.4. 什么是“嵌入式开发”

	电脑软件开发	嵌入式开发
运行环境	操作系统（如 Windows / Linux）负责管理硬件，并提供任务调度功能	通常没有完整操作系统，程序直接跑在芯片上，通常只有一个线程
软硬件关系	极度解耦，有强大的OS和虚拟机保证软件的通用性，编程时无需关心硬件	代码和硬件是“血肉相连”的，只懂硬件或者只懂软件都不行
硬件资源	CPU、内存、硬盘都很强大且丰富	CPU主频低，内存很小，性能有限
输入输出	键盘、鼠标、屏幕	引脚（GPIO）、串口、传感器、LED、电机等
程序启动方式	双击或命令运行，操作系统加载	上电后CPU自动从Flash读取程序执行
程序终止方式	程序运行完成成后自动终止或用户手动关闭	掉电终止

02. ESP32-S3简介



2.1. ESP32-S3简介



乐鑫信息科技 (Espressif Systems) (国产) 推出

esp=乐鑫科技的前三个字母, 32=32位处理器, S3=S系列 (高性能系列) 第三代

目前物联网开发的顶流MCU

特点:

1. 强劲性能: 双核处理器, 向量指令集带来强大的AI性能
2. 物联网功能: 集成天线、Wi-Fi、蓝牙模块, 可以联网, 也可以与手机等设备无限通讯
3. 丰富的外设: 可以直接连接多种传感器、电机、摄像头、SD卡等设备

适用领域: 物联网、智能家居、机器人

2.2. ESP32-S3核心参数速览



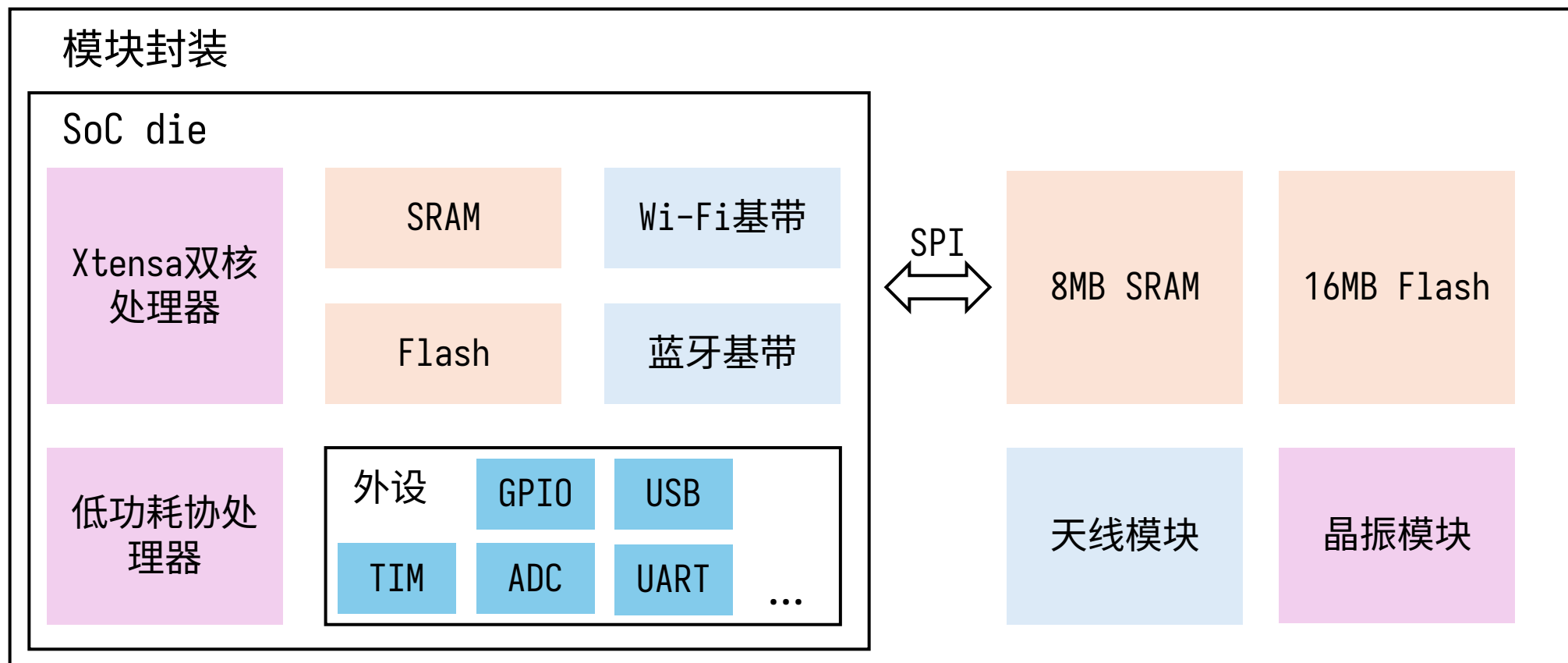
ESP32-S3-WROOM-1-N16R8

WROOM: 包含天线;

N16R8: 封装16MB Flash和8MB PSRAM

项目	参数
内核	Xtensa 32位LX7双核处理器（带FPU和向量单元）+ 低功耗协处理器
主频	240 MHz
Flash	384KB片内集成+16MB外部封装
SRAM	512 KB片内集成+8MB外部封装
Wi-Fi	2.4 GHz 频带，20 MHz 和40 MHz 频宽
蓝牙	Bluetooth 5
GPIO	共45个GPIO，其中36个引出
ADC	2×ADC，20×通道
定时器	4×定时器 + 3×看门狗定时器
通信接口	3×UART、2×I ² C、2×SPI、1×USB、1×DVP摄像头接口
工作电压	3.0 V - 3.6 V（常用3.3V）

2.3. ESP32-S3系统架构



2.4. 如何查找官方资料

在“乐鑫科技官网/支持/技术文档”上搜索芯片型号，可以免费获得大量官方资料

手册类型	内容	建议内容
Datasheet（技术规格书）	芯片参数、电气特性、引脚定义	快速了解芯片整体特性
Reference Manual（技术参考手册）	各功能和外设寄存器的详细说明（GPIO、USART、ADC 等）	学习寄存器或深入理解外设
Arduino ESP32 Document（API文档）	API定义，编程示例	快速了解如何编程

基础编程选择Arduino ESP32 Document
深入学习硬件架构选择Reference Manual
画板选择Datasheet

03. GPIO的原理和使用



3.1. GPIO简介及其功能

GPIO=General Purpose Input Output

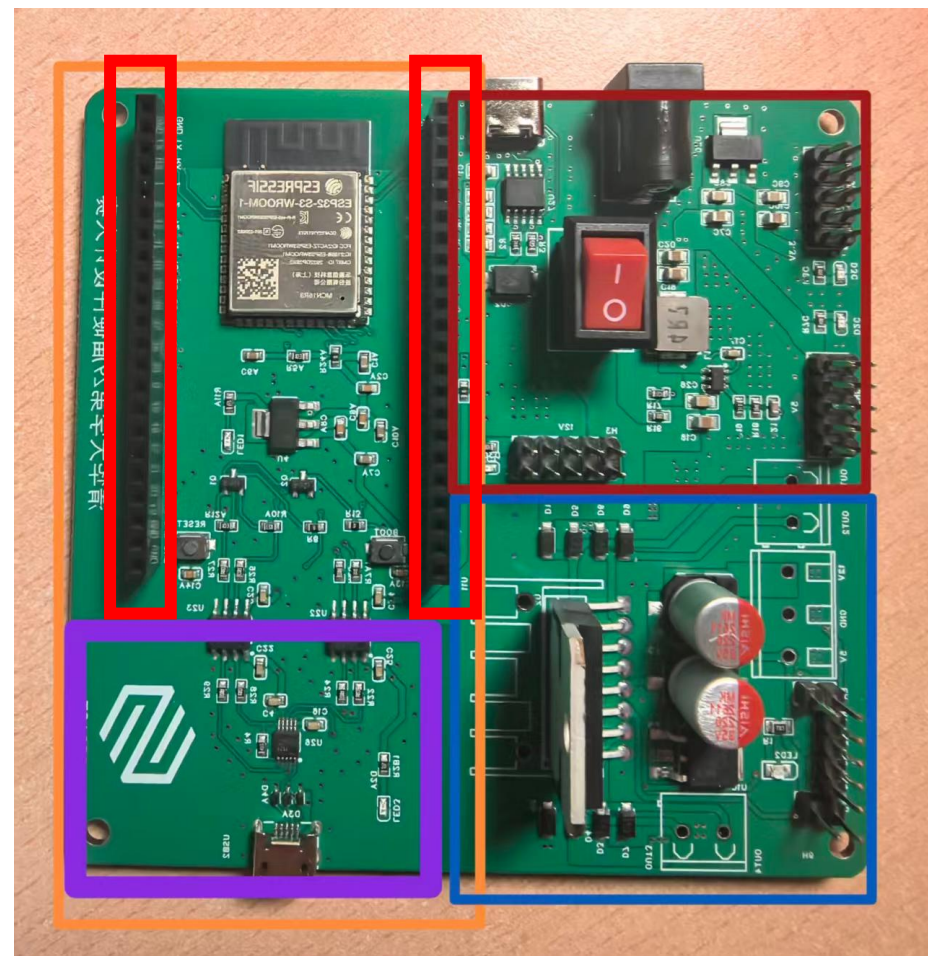
GPIO的功能包括：

1. 通用输入输出（可由程序直接控制）
2. 复用输入输出（连接其他外设，由程序间接控制）

开发板中引出的GPIO引脚：0-21，35-48

有特殊用途而不建议使用的引脚：

1. 影响MCU正常启动：0，3，45，46
2. USB串口：19，20
3. JTAG：39-42
4. 烧录端口：43，44



3.2. GPIO硬件结构

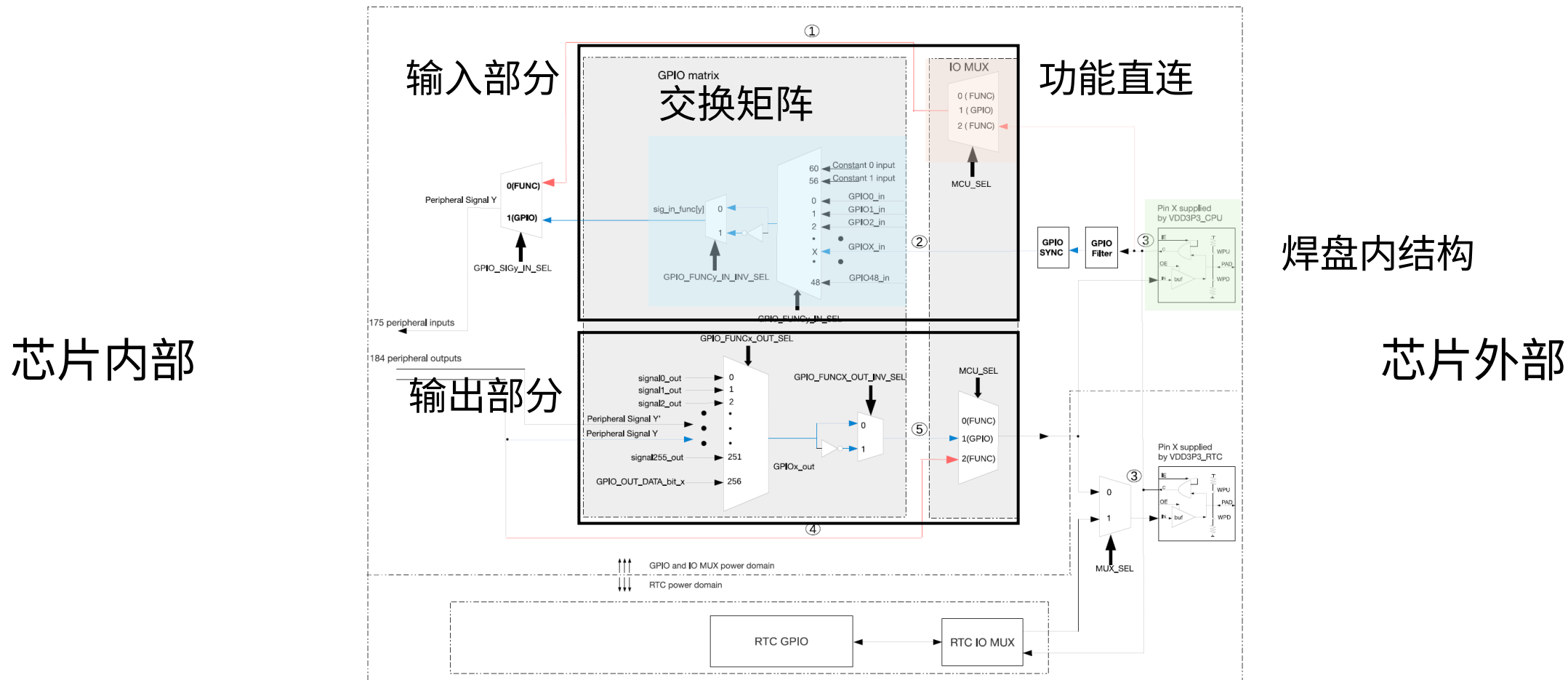


图 6.3-1. IO MUX、RTC IO MUX 和 GPIO 交换矩阵结构框图

3.2. GPIO硬件结构

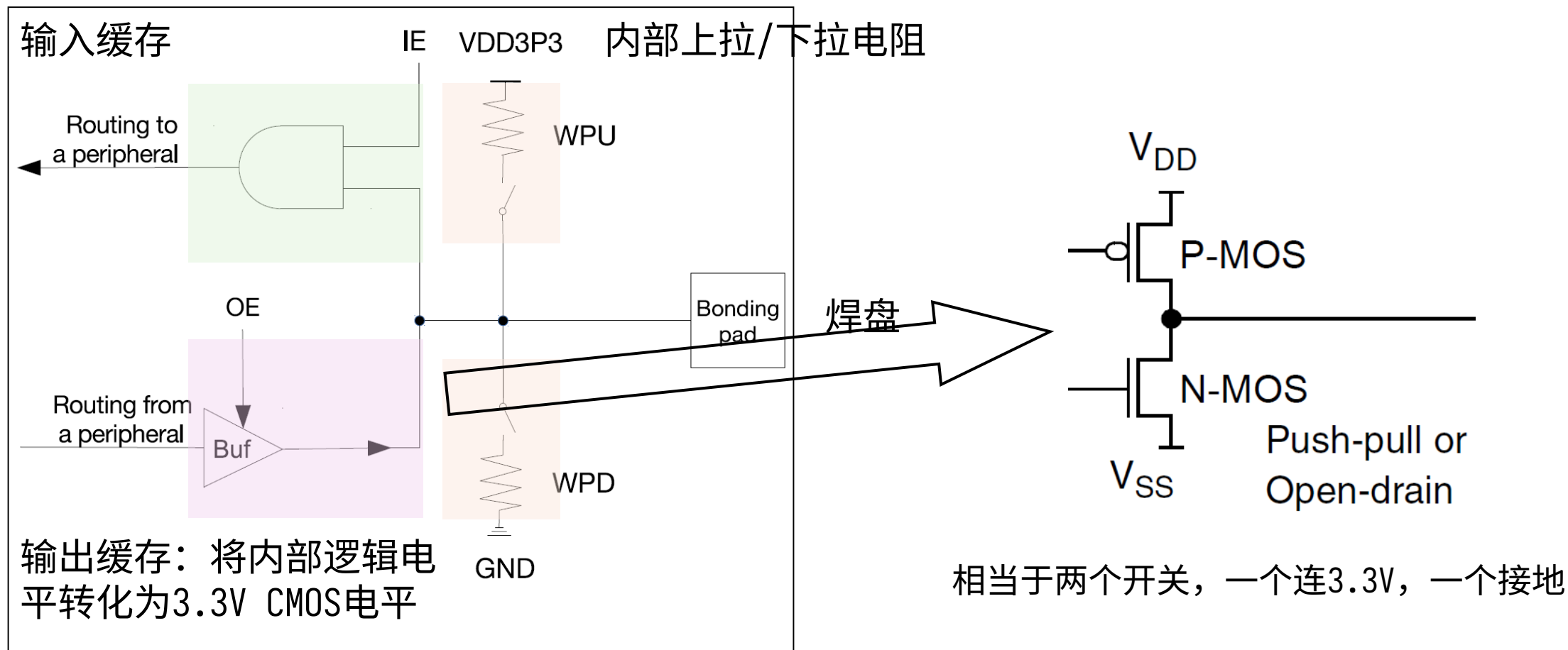
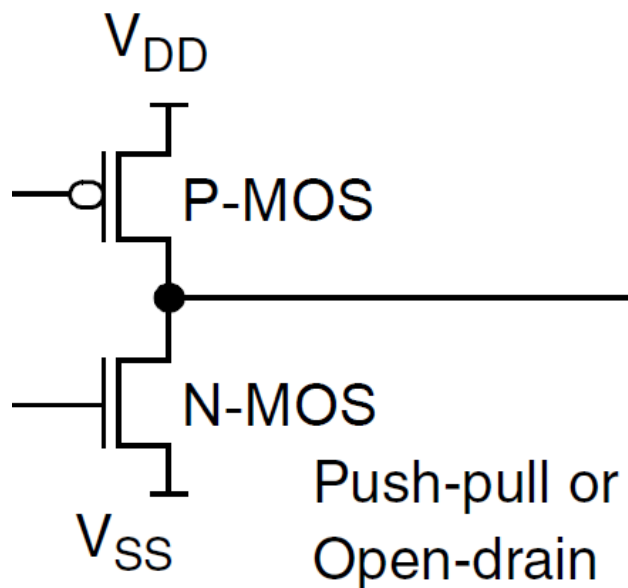


图 6.3-2. 焊盘内部结构

3.3. GPIO的工作模式

通用输入输出：数字输出模式



1. 推挽输出

两个开关总是一开一关，内部电源直接驱动

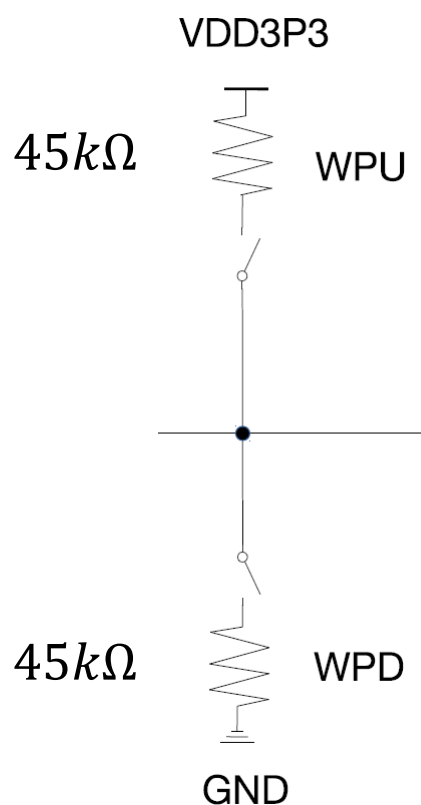
2. 开漏输出

上面的开关总是断开，下面的开关时开时关。需要外部连接上拉电阻，用于驱动高电压设备

3. 高阻态（不输出）

两个开关总是断开，此时相当于断路，不影响输入功能

3.3. GPIO的工作模式



通用输入输出：数字输入模式

1. 直接输入

上拉/下拉电阻均断开，此时引脚内部相当于断路

问题：悬空时电平不确定

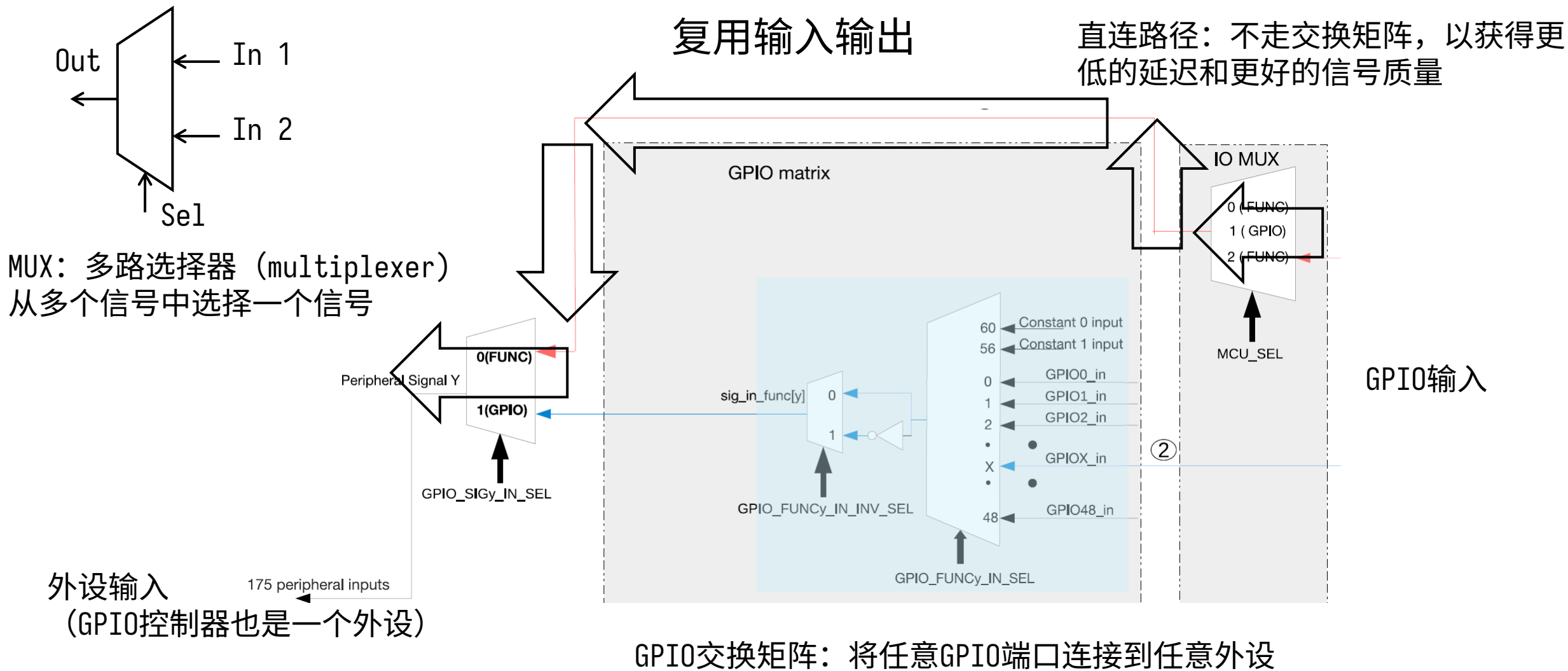
2. 上拉输入

上拉电阻连通，悬空时读到高电平

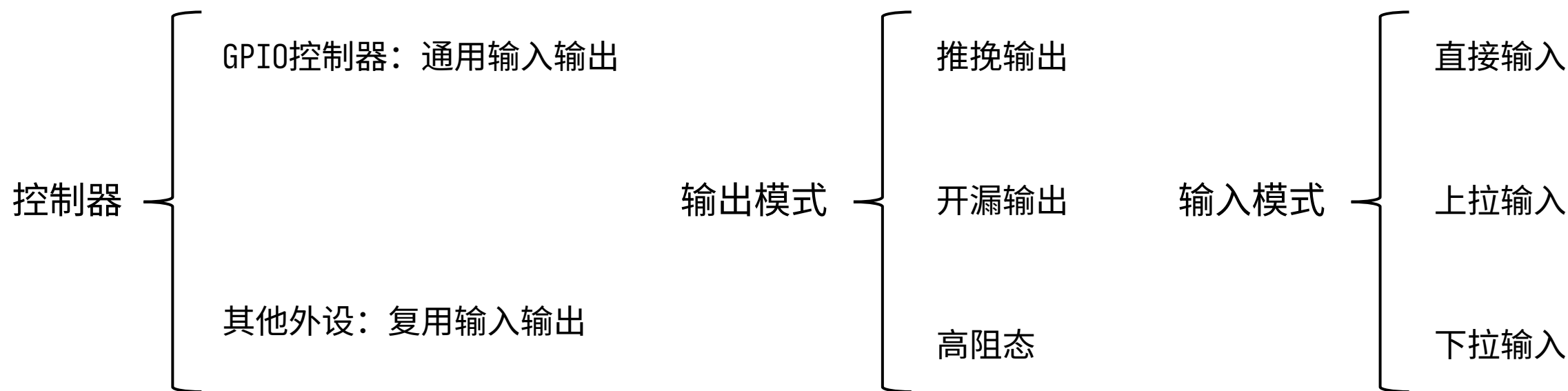
3. 下拉输入

下拉电阻连通，悬空时读到低电平

3.3. GPIO的工作模式



3.3. GPIO的工作模式



3.4. GPIO的直流特性

表 6-3. 直流电气特性 (3.3 V, 25 °C)

参数	说明	最小值	典型值	最大值	单位
C_{IN}	管脚电容	—	2	—	pF
V_{IH}	高电平输入电压	$0.75 \times VDD^1$	—	$VDD^1 + 0.3$	V
V_{IL}	低电平输入电压	-0.3	—	$0.25 \times VDD^1$	V
I_{IH}	高电平输入电流	—	—	50	nA
I_{IL}	低电平输入电流	—	—	50	nA
V_{OH}^2	高电平输出电压	$0.8 \times VDD^1$	—	—	V
V_{OL}^2	低电平输出电压	—	—	$0.1 \times VDD^1$	V
I_{OH}	高电平拉电流 ($VDD^1 = 3.3\text{ V}$, $V_{OH} \geq 2.64\text{ V}$, PAD_DRIVER = 3)	—	40	—	mA
I_{OL}	低电平灌电流 ($VDD^1 = 3.3\text{ V}$, $V_{OL} = 0.495\text{ V}$, PAD_DRIVER = 3)	—	28	—	mA
R_{PU}	内部弱上拉电阻	—	45	—	k Ω
R_{PD}	内部弱下拉电阻	—	45	—	k Ω
V_{IH_nRST}	芯片复位释放电压 (EN 管脚应满足电压范围)	$0.75 \times VDD^1$	—	$VDD^1 + 0.3$	V
V_{IL_nRST}	芯片复位电压 (EN 管脚应满足电压范围)	-0.3	—	$0.25 \times VDD^1$	V

小“寄”巧

错误做法	后果	正确做法
使用GPIO输出驱动高电平的器件	GPIO通常输出3.3V，高电平器件可能无法识别	使用电平转换芯片
使用GPIO输入读取高电平的信号	GPIO最多输入3.6V，超过会导致不可逆的损坏	使用电平转换芯片，低速情况下可以使用电阻分压
使用GPIO驱动电机、舵机、音响等大功率器件	GPIO输出功率较低，无法驱动。感性负载可能产生高压反向电动势，烧毁器件	外部连接大功率驱动电路，GPIO控制驱动电路

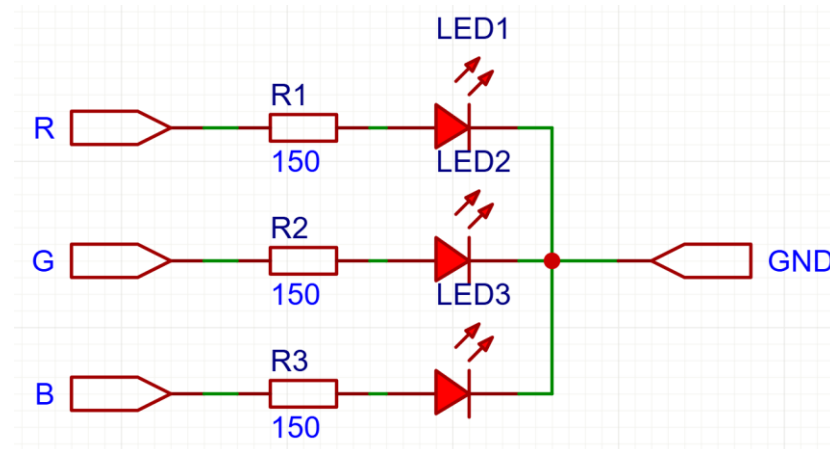
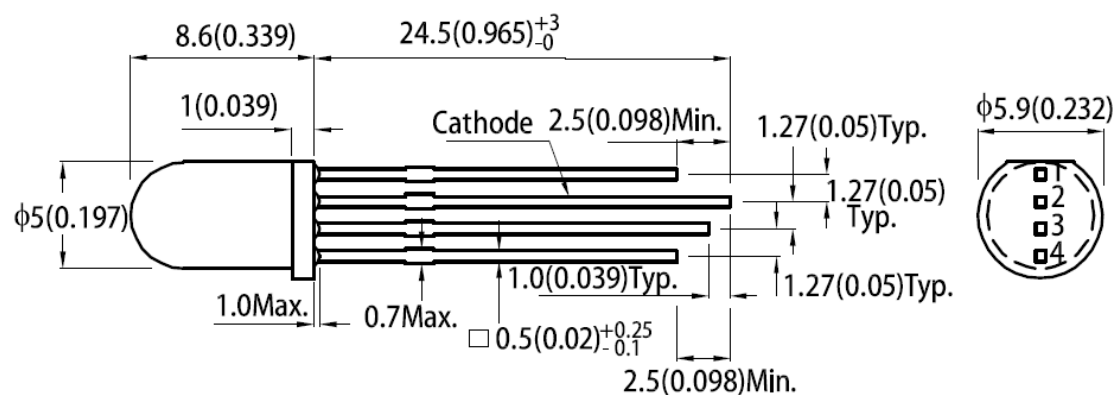
3.5. 实验一

GPIO点灯实验

实验内容：使用寄存器，点亮第一颗LED

学习目标：理解软件控制硬件的基本原理

3.5. 实验一



Forward Voltage $I_F = 20\text{mA}$	$V_F^{[2]}$	Hyper Red	1.9	2.5	V
		Blue	3.3	4.0	
		Green	3.3	4.1	

3.3V和5V均可点亮

‘-’连接开发板上的GND

R, G, B任选其一连接开发板上的GPIO

3.5. 实验一

```
void setup()
{
    // put your setup code here, to run once:
}

void loop()
{
    // put your main code here, to run repeatedly:
}
```

Arduino IDE的代码模板

setup()中应包含初始化代码，上电后
执行一次

loop()中的代码会循环执行，直到断电

```
int main()
{
    setup();

    while (1)
    {
        loop();
    }
}
```

3.5. 实验一*

需要使用的寄存器：

6.14 寄存器列表

6.14.1 GPIO 交换矩阵寄存器列表

本小节的所有地址均为相对于 GPIO 基地址的地址偏移量（相对地址），具体基地址请见章节 4 系统和存储器 中的表 4.3-3。

Register 6.10. GPIO_ENABLE_W1TS_REG (0x0024)

GPIO_ENABLE_W1TS	
31	0
0x000000	
Reset	

GPIO输出使能置位寄存器

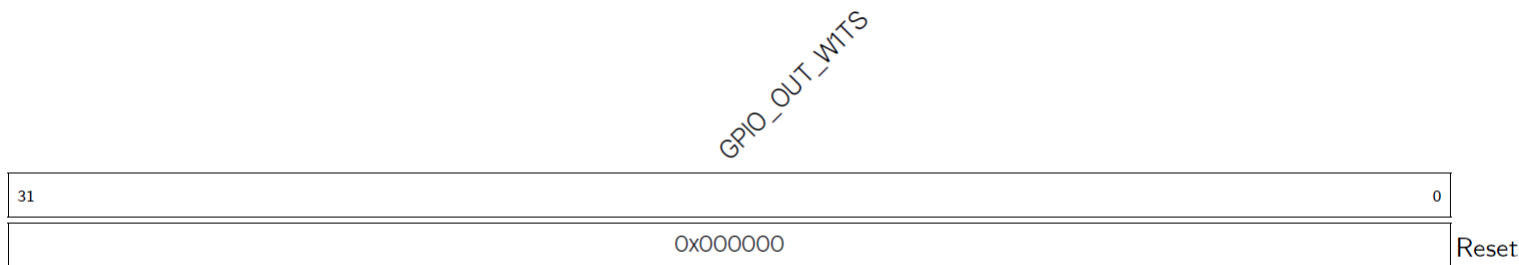
共32位，从低到高依次代表GPIO0-31

将对应位写入1，则硬件电路会将对应的GPIO设置为通用推挽输出模式

GPIO_ENABLE_W1TS GPIO0 ~ 31 输出使能置位寄存器。每一位置 1，则 **GPIO_ENABLE_REG** 中的相应位也置 1。注：推荐使用此寄存器来置位 **GPIO_ENABLE_REG**。(WO)

3.5. 实验一*

Register 6.3. GPIO_OUT_W1TS_REG (0x0008)



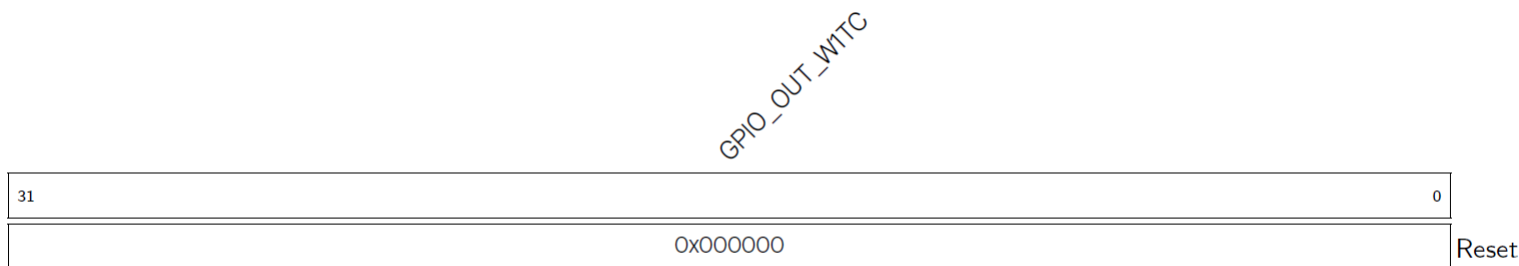
GPIO_OUT_W1TS GPIO00 ~ 31 输出置位寄存器。每一位置 1, **GPIO_OUT_REG** 中的相应位也置 1。注：推荐使用此寄存器来置位 **GPIO_OUT_REG**。(WO)

GPIO输出置位寄存器

共32位，从低到高依次代表GPIO0-31

将对应位写入1，则硬件电路会将对应的GPIO设置为输出高电平

Register 6.4. GPIO_OUT_W1TC_REG (0x000C)



GPIO_OUT_W1TC GPIO00 ~ 31 输出清零寄存器。每一位置 1, 则 **GPIO_OUT_REG** 中的相应位会清零。注：推荐使用此寄存器来清零 **GPIO_OUT_REG**。(WO)

GPIO输出清零寄存器

共32位，从低到高依次代表GPIO0-31

将对应位写入1，则硬件电路会将对应的GPIO设置为输出低电平

3.6. 实验二

GPIO闪灯实验

实验内容：让LED灯以一定频率闪烁

学习目标：学习Arduino IDE的基本使用方法，理解GPIO的推挽输出模式

3.6. 实验二

pinMode The `pinMode` function is used to define the GPIO operation mode for a specific pin.

```
void pinMode(uint8_t pin, uint8_t mode);
```

- `pin` defines the GPIO pin number.
- `mode` sets operation mode.

The following modes are supported for the basic *input* and *output*:

- **INPUT** sets the GPIO as input without pullup or pulldown (high impedance).
- **OUTPUT** sets the GPIO as output/read mode.
- **INPUT_PULLDOWN** sets the GPIO as input with the internal pulldown.
- **INPUT_PULLUP** sets the GPIO as input with the internal pullup.

digitalWrite The function `digitalWrite` sets the state of the selected GPIO to HIGH or LOW. This function is only used if the `pinMode` was configured as OUTPUT.

```
void digitalWrite(uint8_t pin, uint8_t val);
```

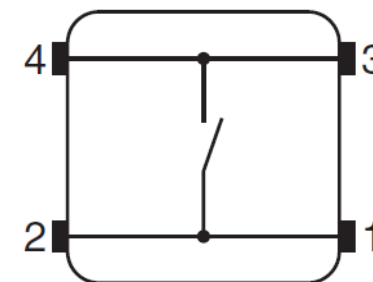
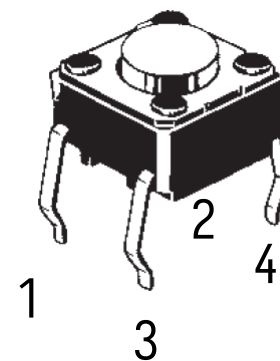
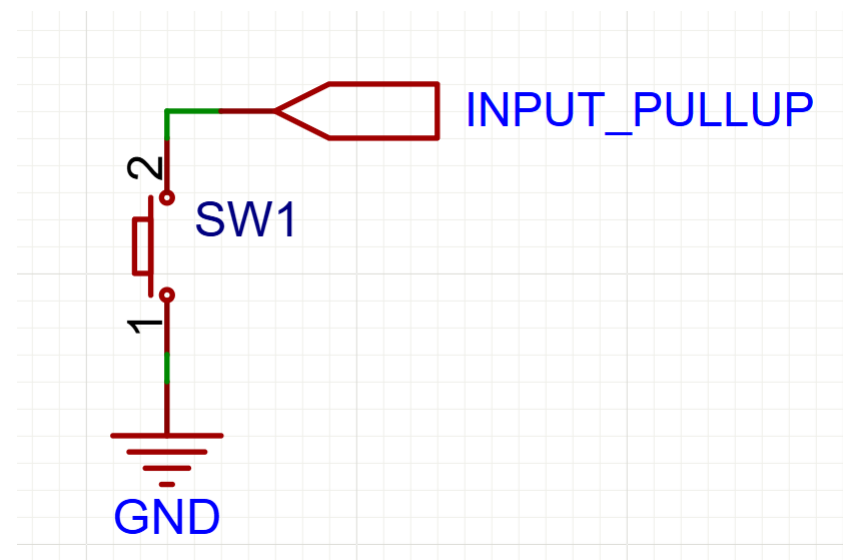
- `pin` defines the GPIO pin number.
- `val` set the output digital state to HIGH or LOW.

3.7. 实验三

GPIO读取按钮

实验内容：按下按钮，LED灯点亮，再次
按下按钮，LED灯熄灭

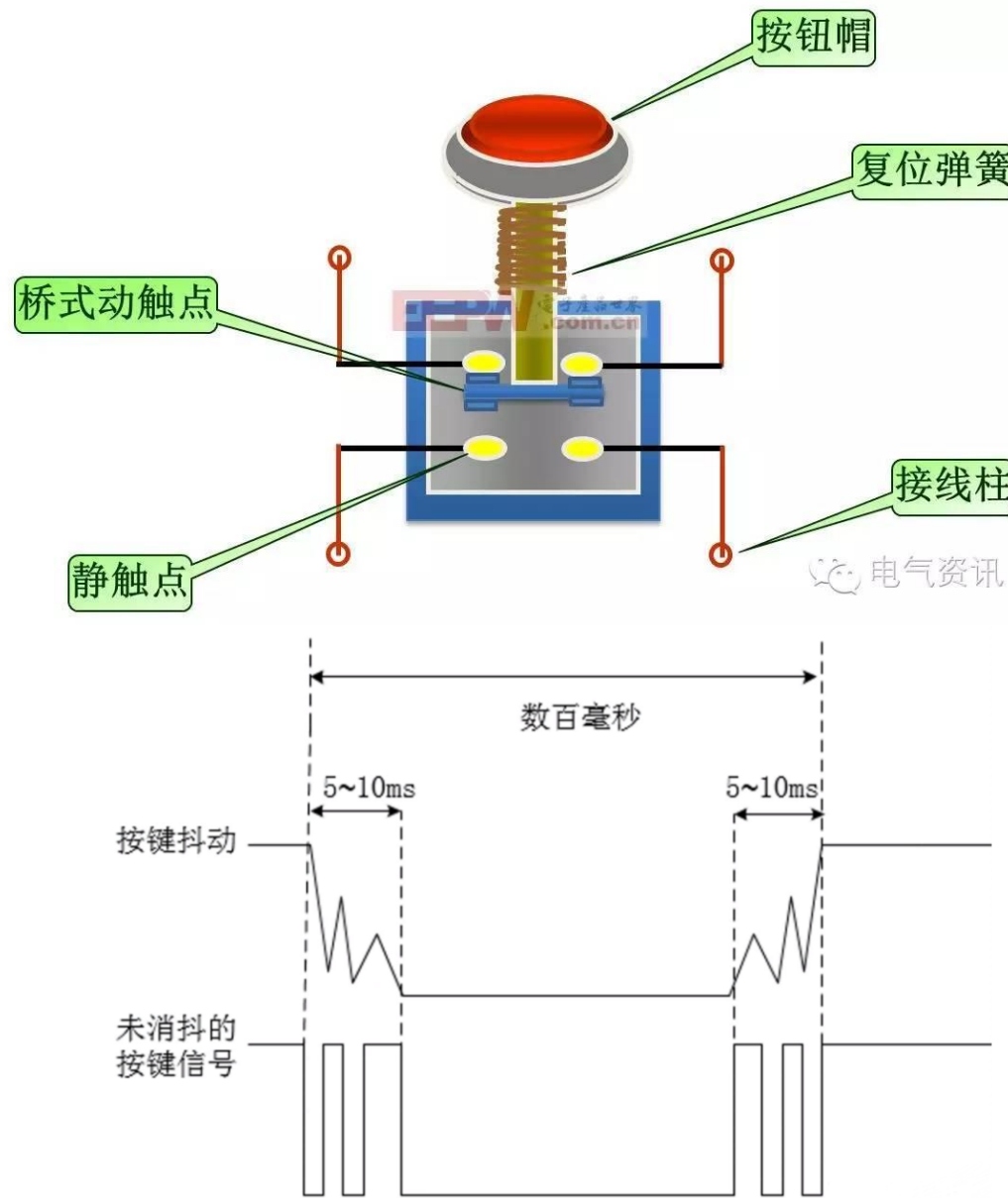
学习目标：理解GPIO的通用输入模式，
学习按钮消抖的方法



3.7. 实验三

按钮消抖

1. 硬件消抖：连接RC低通滤波器，效果好，但是需要付出硬件成本
2. 软件消抖：检测到电平跳变后，等待一段时间，再次确认电平是否稳定，如果稳定，则认为按钮按下



04. 中断的原理和使用



4.1. 中断的作用

场景一：

按下按钮，LED灯开始以5秒为周期闪烁；再次按下按钮，LED灯停止闪烁。

实现方法：

在loop()中，每次循环检测按钮是否按下。

问题：

假如恰好在执行delay函数时按下，应该如何检测？

```
void loop()
{
    if (button_pressed)
        led_en = !led_en;

    if (led_en)
    {
        digitalWrite(LED_PIN, HIGH);
        delay(2500);
        digitalWrite(LED_PIN, LOW);
        delay(2500);
    }
    else
    {
        digitalWrite(LED_PIN, LOW);
    }
}
```

4.1. 中断的作用

场景二：

一边烧水一边打游戏，水一烧开就需要立即把火关掉。

实现方法：

每隔一分钟，去检查一下水是否烧开。

问题：

频繁地检查水是否烧开导致游戏掉分。

4.1. 中断的作用

场景一：

按下按钮，LED灯开始以5秒为周期闪烁；再次按下按钮，LED灯停止闪烁。

修正方案：

正常情况下只处理LED灯的闪烁，不去管按钮。按钮按下时才切换状态。

场景二：

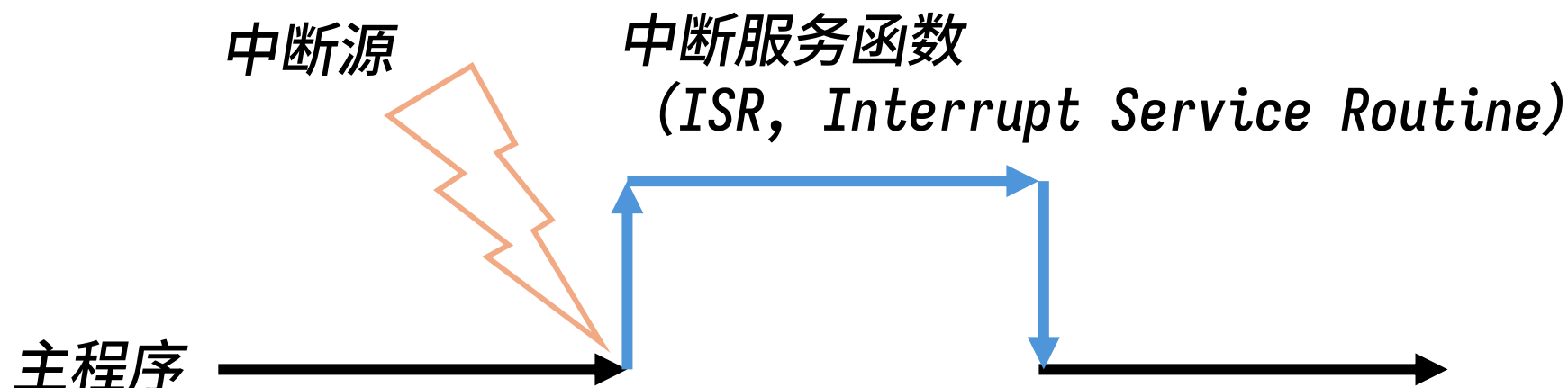
一边烧水一边打游戏，水一烧开就需要立即把火关掉。

修正方案：

正常情况下专著打游戏，听到水烧开的声音后立即停止游戏，把火关掉。

轮询 (polling) -> 中断 (interrupt)

4.2. 中断机制



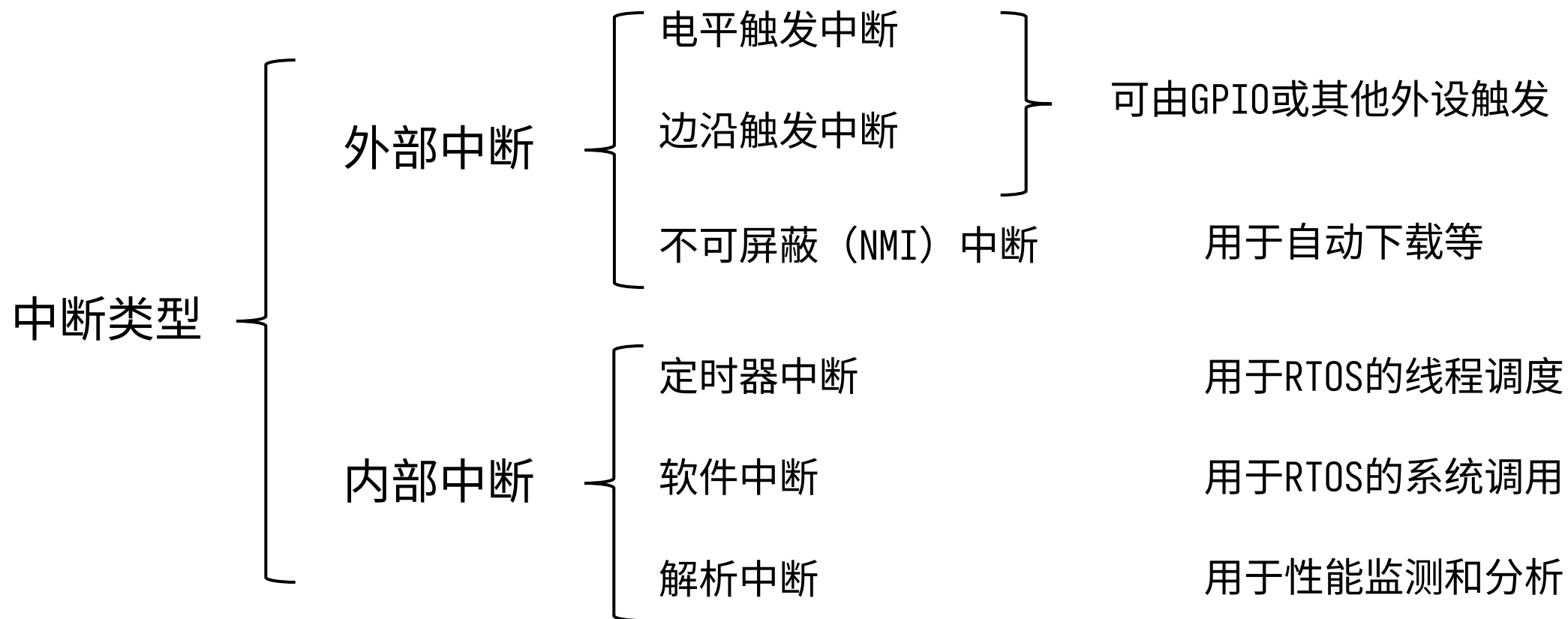
正常情况下执行主程序，中断源到来时，处理器立即保存当前状态，并开始执行对应的中断服务函数。中断服务函数完成后，处理器恢复中断前状态，继续执行主程序。

4.2. 中断机制

中断无处不在

场景	主程序	中断源	中断服务函数
用电脑看视频时，突然动了一下鼠标	视频解码、UI渲染	USB管理器或蓝牙模块	重新计算鼠标位置
正在打游戏时，突然触发了低电量警告	游戏逻辑、画面渲染	电池管理模块	低电量报警函数
电脑蓝屏，按电源键强制关机	卡死的操作系统	电源按钮	强制关机程序

4.3. ESP32-S3的中断机制



4.3. ESP32-S3的中断机制

外部中断触发类型：

1. DISABLED：关闭中断
2. RISING：上升沿触发
3. FALLING：下降沿触发
4. CHANGE：边沿触发，上升沿和下降沿均可
5. ONLOW：低电平触发，处理器响应前需要保持电平，否则会丢失中断
6. ONHIGH：高电平触发，处理器响应前需要保持电平
7. ONLOW_WE：低电平触发，同时将处理器从休眠模式中唤醒，处理器响应前需要保持电平
8. ONHIGH_WE：高电平触发，同时将处理器从休眠模式中唤醒，处理器响应前需要保持电平

4.4. 使用Arduino API绑定外部中断

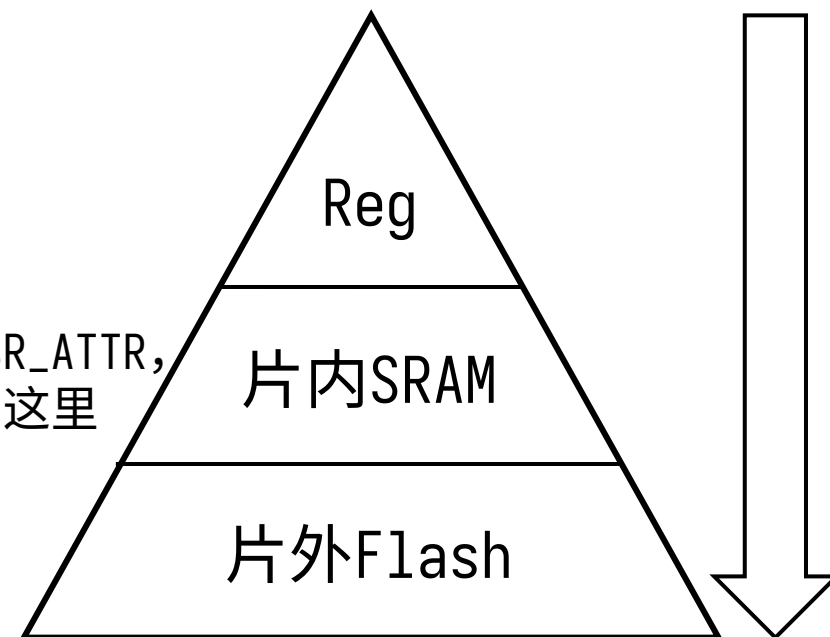
Step 1. 使用 pinMode 将对应引脚设置为输入

Step 2. 创建中断服务函数，并使用ARDUINO_ISR_ATTR修饰

```
void ARDUINO_ISR_ATTR isr(void *argv)
{
    // interrupt service routine
}
```

使用ARDUINO_ISR_ATTR,
强制将函数存在这里

默认情况下，程
序存储在这里



加快ISR的读取速度，提升响应及时性。
如果主程序正在写入片外Flash，则片外Flash将临时不可读。

4.4. 使用Arduino API绑定外部中断

Step 1. 使用 `pinMode` 将对应引脚设置为输入

Step 2. 创建中断服务函数，并使用 `ARDUINO_ISR_ATTR` 修饰

Step 3. 使用 `attachInterrupt` 绑定端口和中断

attachInterrupt The function `attachInterrupt` is used to attach the interrupt to the defined pin.

```
attachInterrupt(uint8_t pin, voidFuncPtr handler, int mode);
```

- `pin` defines the GPIO pin number.
- `handler` set the handler function.
- `mode` set the interrupt mode.

`pin`: 需要绑定的GPIO端口

`handler`: 函数指针，指向中断服务函数

`mode`: 中断服务类型，可以是 `RISING`, `CHANGE`, `ONLOW` 等

如果需要向ISR传入参数，
可以使用 `attachInterruptArg`

4.4. 使用Arduino API绑定外部中断

注意事项:

1. 中断服务函数应该尽可能简短，避免使用 `delay()`, `Serial.println()` 等耗时较长的函数，避免进行复杂运算，以免导致主程序卡顿导致时序问题。
2. 对于主程序和中断程序同时使用的变量，需要使用 `volatile` 修饰。

```
int flag = 0;

void ARDUINO_ISR_ATTR isr(void *argv)
{
    flag = !flag;
}

void loop()
{
    if (flag)
    {
        // do sth.
    }
}
```

编译器看到 `isr` “不会”被调用，认为 `flag` 不会被修改，因此把它优化为常量或寄存器变量。
`volatile` 表明此变量随时可能被硬件或中断修改，每次读取都强制从内存中读取最新值。

4.5. 实验四

外部中断实验

实验内容：按下按钮，LED灯在闪烁和熄灭之间切换

学习目标：理解中断机制，掌握中断服务函数的绑定方法，理解‘volatile’的作用

谢谢大家

